CE802 2018 - Worked Solutions

Machine Learning and Data Mining

This document gives explanation-first solutions to every question in the uploaded 2018 CE802 paper. Where a numerical answer is required, the result is derived step by step from the relevant formula rather than presented as a bare answer.

Conventions used below: entropy is measured in bits; for Apriori, support is shown both as a transaction count and as a proportion when helpful; and for the reinforcement-learning question, terminal states are assigned value 0 because rewards are attached to the incoming transitions shown in the figure.

Paper structure	Question 1: decision trees and information gain Question 2: Markov decision processes and discounted return Question 3: association rules and Apriori
Working style	Each sub-question shows the governing formula, the intermediate computation, and the interpretation needed for exam-quality reasoning.

Question 1 - Decision Trees

1(a) Information gain for Colour, Type and Material

There are 12 training instances. The target variable *Sales Increase* contains 6 Yes and 6 No, so the root entropy is

```
H(S) = - (6/12) log2(6/12) - (6/12) log2(6/12) = 1 bit.
```

For an attribute A, the information gain is

```
IG(S, A) = H(S) - sum_v (|S_v|/|S|) H(S_v).
```

Attribute	Child distributions	Weighted child entropy	Information gain
Colour	White: 3 Yes, 3 No -> H = 1 Green: 3 Yes, 3 No -> H = 1	(6/12)*1 + (6/12)*1 = 1.0000	1.0000 - 1.0000 = 0.0000
Type	Indoor: 5 Yes, 4 No -> H = 0.9911 Outdoor: 1 Yes, 2 No -> H = 0.9183	(9/12)*0.9911 + (3/12)*0.9183 = 0.9729	1.0000 - 0.9729 = 0.0271
Material	Wood: 3 Yes, 3 No -> H = 1 Plastic: 3 Yes, 3 No -> H = 1	(6/12)*1 + (6/12)*1 = 1.0000	1.0000 - 1.0000 = 0.0000

Therefore the greedy choice at the root is **Type**, because it is the only attribute with positive information gain.

1(b) Constructing the complete decision tree

After splitting on **Type**, the **Outdoor** branch contains three examples: two No and one Yes. Using the remaining attributes, **Material** separates this branch perfectly: Outdoor + Plastic -> Yes, and Outdoor + Wood -> No.

The **Indoor** branch contains 9 examples (5 Yes, 4 No). We compare the remaining two attributes there:

```
IG(Indoor, Colour) = H(5/9,4/9) - [ (3/9) H(2/3,1/3) + (6/9) H(3/6,3/6) ]
= 0.9911 - [ (3/9)*0.9183 + (6/9)*1 ]
= 0.0183.

IG(Indoor, Material) = H(5/9,4/9) - [ (5/9) H(2/5,3/5) + (4/9) H(3/4,1/4) ]
= 0.9911 - [ (5/9)*0.9710 + (4/9)*0.8113 ]
= 0.0911.
```

So inside the Indoor branch, the next greedy split is **Material**.

Subset	Examples	Best next split	Resulting leaves
Indoor & Wood	3 Yes, 1 No	Colour	White -> Yes, Green -> No
Indoor & Plastic	2 Yes, 3 No	Colour	Green -> Yes, White -> No
Outdoor	1 Yes, 2 No	Material	Plastic -> Yes, Wood -> No

A compact way to display the final greedy tree is:

```
Type?
|- Outdoor -> Material?
|   |- Plastic -> Yes
|   `- Wood    -> No
`- Indoor  -> Material?
    |- Wood    -> Colour?  White -> Yes, Green -> No
    `- Plastic -> Colour?  Green -> Yes, White -> No
```

This tree classifies all 12 training examples correctly.

1(c) Why a lower-gain first split can produce a shallower tree, and a modified algorithm

A key interaction exists between **Colour** and **Material**. Although each one has zero information gain on its own at the root, the pair together determines the class exactly:

Colour	Material	Class
White	Wood	Yes

Colour	Material	Class
White	Plastic	No
Green	Plastic	Yes
Green	Wood	No

Hence a shallower tree is obtained by choosing either **Colour** or **Material** at the root and the other one immediately after. For example, **Material -> Colour** gives depth 2, which is shallower than the greedy tree above (maximum depth 3).

A modified induction algorithm can exploit this by using a **two-step look-ahead**: at each node, evaluate every ordered pair of remaining attributes (A, B), score the weighted impurity after splitting first on A and then on B inside every branch, and choose the pair that produces the best two-level reduction in impurity. The tree can then expand by creating the first split and reserving the second split for its children, or by creating a compound split directly.

The main limitation is computational cost and data fragmentation. If there are m candidate attributes, greedy ID3/C4.5 style selection evaluates O(m) choices per node, whereas pairwise look-ahead evaluates $O(m^2)$ pairs. With high-cardinality attributes or small datasets, many second-level branches may contain very few examples, making the estimated entropies noisy and the procedure less reliable.

Question 2 - Markov Decision Processes

2(a) Meaning of an MDP and of the tuple <S, A, T, R>

A Markov Decision Process is a model for sequential decision-making under uncertainty. It satisfies the Markov property: once the current state is known, the future depends only on that state and the chosen action, not on the earlier history.

Component	Meaning in general	Meaning here
S	Set of states	Towns / positions in the transition diagram: A, B, C, D, E, G1, G2, G3
A(s)	Actions available in state s	The arrows leaving each state; for example, from D the available moves are to C and to G1
T(s'|s,a)	Transition probability	Deterministic here: each chosen action leads to one specific next state with probability 1
R(s,a)	Immediate reward for taking action a in state s	Only the labeled transitions give non-zero reward: D->G1 gives 60, E->G2 gives 50, G2->G1 gives 60, and G2->G3 gives 70

Because transitions are deterministic, the Bellman backups simplify to 'reward + discounted value of the unique next state'.

2(b) Interpreting the Bellman optimality equation

```
V*(s) = max_{a in A(s)} [ R(s,a) + gamma V*(s') ]
```

This equation says: the optimal value of a state s is the best achievable discounted return starting from s. For each available action a, compute the immediate reward R(s,a) plus the discounted value of the next state s'. Then choose the action that makes this quantity largest. The discount factor gamma in [0,1] controls how strongly future rewards influence the present decision.

2(c) Discounted cumulative value of each state for gamma = 0.5

From Figure 2.1 on page 3 of the uploaded paper, the deterministic transitions are: A->D; D->C or G1; C->D, E or B; B->C; E->C, B or G2; G2->G1 or G3. Rewards are attached to D->G1 (60), E->G2 (50), G2->G1 (60), and G2->G3 (70). The terminal states G1 and G3 have value 0 because no further reward is available after entering them.

```
V*(G1) = 0, V*(G3) = 0.
V*(G2) = max(60 + 0.5 V*(G1), 70 + 0.5 V*(G3)) = max(60, 70) = 70.
V*(E) = max(0.5 V*(C), 0.5 V*(B), 50 + 0.5 V*(G2)) = max(0.5 V*(C), 0.5 V*(B), 85).
V*(D) = max(0.5 V*(C), 60 + 0.5 V*(G1)) = max(0.5 V*(C), 60).
V*(B) = 0.5 V*(C).
V*(C) = max(0.5 V*(D), 0.5 V*(E), 0.5 V*(B)).
V*(A) = 0.5 V*(D).
```

Now solve from the states with known best options. Since 85 is already larger than any plausible looping alternative, E optimally goes to G2, so V*(E)=85. Then

```
V*(D) = max(0.5 V*(C), 60).
V*(C) = max(0.5 V*(D), 0.5*85, 0.5 V*(B)) = max(0.5 V*(D), 42.5, 0.5 V*(B)).
V*(B) = 0.5 V*(C).
```

Because 42.5 already exceeds 0.5*60 = 30, the best move from C is to go to E, and hence

```
V*(C) = 42.5,
V*(B) = 0.5 * 42.5 = 21.25,
V*(D) = max(0.5 * 42.5, 60) = 60,
V*(A) = 0.5 * 60 = 30.
```

State	Optimal value V*(s)	Best action / interpretation
A	30	Move to D, then take the immediate-reward route from D
B	21.25	Move to C
C	42.5	Move to E

State	Optimal value V*(s)	Best action / interpretation
D	60	Go directly to G1
E	85	Go directly to G2
G2	70	Go to G3 rather than G1
G1	0	Terminal
G3	0	Terminal

2(d) Role of the discount factor and the policy switch

The discount factor decides whether the agent prefers earlier, smaller rewards or later, possibly larger rewards. From state D there are two qualitatively different strategies:

```
Immediate route: D -> G1, return = 60.
Delayed route: D -> C -> E -> G2 -> G3,
return = 50 gamma^2 + 70 gamma^3.
```

The policy switches when these two returns are equal:

```
60 = 50 gamma^2 + 70 gamma^3.
```

The solution in [0,1] is approximately **gamma = 0.762**. Therefore: for gamma below about 0.762, the agent prefers the immediate reward 60 at G1; for gamma above about 0.762, it becomes worthwhile to wait for the larger delayed rewards available through E and G2.

State A has only one outgoing action (to D), but the optimal *plan starting from A* still changes indirectly with gamma because the downstream decision at D changes. This is exactly the role of discounting: it translates 'later' into 'worth less now', and may therefore alter the optimal policy even when all rewards are positive.

Question 3 - Association Rules and Apriori

3(a) Definitions

Measure	Probabilistic definition	Interpretation
Confidence	conf(X => Y) = P(Y|X) = support(X union Y) / support(X)	How often the rule is correct when its antecedent is present.
Support	support(X) = P(X)	Fraction (or count) of transactions containing X.
Lift	lift(X => Y) = P(Y|X) / P(Y) = support(X union Y) / (support(X) support(Y))	How much more often X and Y occur together than expected under independence.

3(b) Apriori with minimum support 20%

There are 9 transactions, so the minimum support count is 0.20 x 9 = 1.8. Therefore an itemset is frequent iff it appears in **at least 2 transactions**.

Candidate 1-itemset	Support count	Retained?
{A}	6	Yes
{B}	6	Yes
{C}	7	Yes
{D}	2	Yes
{E}	2	Yes
{F}	1	No - insufficient support
{G}	1	No - insufficient support

Thus L1 = { {A}, {B}, {C}, {D}, {E} }. Items F and G are removed immediately.

Candidate 2-itemset	Support count	Decision / reason
{A,B}	4	Retained in L2
{A,C}	4	Retained in L2
{A,D}	1	Discarded - insufficient support
{A,E}	2	Retained in L2
{B,C}	4	Retained in L2
{B,D}	0	Discarded - insufficient support
{B,E}	1	Discarded - insufficient support
{C,D}	2	Retained in L2
{C,E}	2	Retained in L2
{D,E}	0	Discarded - insufficient support

Therefore L2 = {AB, AC, AE, BC, CD, CE}.

Candidate 3-itemset	All 2-subsets frequent?	Support count	Decision
{A,B,C}	Yes	2	Retained in L3
{A,C,E}	Yes	2	Retained in L3
{A,B,E}	No (BE not frequent)	-	Pruned by Apriori property
{A,C,D}	No (AD not frequent)	-	Pruned by Apriori property
{B,C,D}	No (BD not frequent)	-	Pruned by Apriori property
{C,D,E}	No (DE not frequent)	-	Pruned by Apriori property

Hence L3 = {ABC, ACE}. No size-4 itemset survives, because the only possible union ABCE contains non-frequent 3-subsets such as ABE and BCE, so it is pruned before support counting.

All frequent itemsets are therefore:
{A}:6, {B}:6, {C}:7, {D}:2, {E}:2
{AB}:4, {AC}:4, {AE}:2, {BC}:4, {CD}:2, {CE}:2
{ABC}:2, {ACE}:2

3(c) Strong rules of the form {X,Y} => {Z} with confidence threshold 75%

Only frequent 3-itemsets can produce rules of the requested form. So we evaluate all 2->1 rules from ABC and ACE.

Rule	Support count of XYZ	Confidence	Lift	Strong?
{A,B} => {C}	2	0.500	0.643	No
{A,C} => {B}	2	0.500	0.750	No
{B,C} => {A}	2	0.500	0.750	No
{A,C} => {E}	2	0.500	2.250	No
{A,E} => {C}	2	1.000	1.286	Yes
{C,E} => {A}	2	1.000	1.500	Yes

Therefore the strong rules are:

```
{A,E} => {C}: confidence = 2/2 = 1.0, lift = 1 / (7/9) = 9/7 ≈ 1.286.
{C,E} => {A}: confidence = 2/2 = 1.0, lift = 1 / (6/9) = 1.5.
```

Both rules have lift greater than 1, so each indicates positive association between antecedent and consequent.

3(d) Why Apriori is cheaper than brute force

A brute-force search over the 7 items {A,B,C,D,E,F,G} would need to consider every non-empty itemset, i.e. $2^7 - 1 = 127$ itemsets, and in principle compute support for each one.

Apriori counted support for only:

```
7 singletons + 10 candidate pairs + 2 candidate triples = 19 support computations.
```

The saving comes from the downward-closure (Apriori) property: if an itemset is infrequent, then every superset of it must also be infrequent. This lets the algorithm

- discard F and G after level 1, so no candidate containing them is ever generated;
- discard AD, BD, BE and DE at level 2, thereby pruning many size-3 supersets without counting support;
- avoid even testing size-4 candidates such as ABCE when one of their 3-subsets is already known to be infrequent.

So Apriori saves work in two ways: it generates far fewer candidates than brute force, and it counts support only for those candidates that survive subset-based pruning. The trade-off is that Apriori still needs multiple database passes - one pass per level - but in sparse market-basket data this is usually far cheaper than enumerating every possible itemset.